Mastering JavaScript Design Patterns in React.js

1. Module Pattern

- The **Module Pattern** is used to group related variables, functions, or components into a single unit. It helps control what is public (accessible) and what stays private (internal).
- This avoids **global variables** and makes code more organized and maintainable.

Used in custom hooks, utility functions and API service files to isolate logic.

Example:

```
const ApiService = (() => {
  const BASE_URL = "https://api.example.com"; // private
  const getUsers = async () => {
    const res = await fetch(`${BASE_URL}/users`);
    return res.json();
  };
  return { getUsers }; // public
})();
 useEffect(() => {
  ApiService.getUsers().then(console.log);
}, []);
```

2. Singleton Pattern

- The **Singleton Pattern** ensures that only one instance of a particular object exists across the app.If you try to create more, it returns the same instance.

Used for Redux store, Context Providers, Firebase App instance or configuration objects.

Example:

```
class Config {
  constructor() {
    if (!Config.instance) {
      this.theme = "light";
      Config.instance = this;
    }
    return Config.instance;
  }
}

export const appConfig = new Config();

import { appConfig } from "./Config";
console.log(appConfig.theme);
```

3. Factory Pattern

- The **Factory Pattern** creates different types of objects or components without exposing the complex creation logic.
- You just call a “factory” function with input, and it gives you the right output.

Used in dynamic UI rendering, component factories, and conditional theming.

Example:

```
function ComponentFactory(type, props) {
  const components = {
    button: <button {...props}>{props.label}</button>,
    input: <input {...props} />,
```

```
    text: <p>{props.text}</p>,
  };
  return components[type] || null;
}

export default function App() {
  return (
    <>
      {ComponentFactory("button", { label: "Click Me" })}
      {ComponentFactory("text", { text: "Hello World!" })}
    </>
  );
}
```

4. Observer Pattern

- In the **Observer Pattern**, one object (the subject) maintains a list of dependents (observers).
- When the subject changes, all observers are notified automatically.

React's entire architecture uses this:

- **useState + useEffect**
- **Context API**
- **Redux subscriptions**

Example:

```
function Counter() {
  const [count, setCount] = useState(0);

  useEffect(() => {
```

```
  console.log("Count updated:", count);
 }, [count]);

 return <button onClick={() => setCount(count + 1)}>Count: {count}</button>;
}
```

5. Proxy Pattern

- The **Proxy Pattern** acts as a middle layer that controls access to another object.
- It can **validate**, **log**, or **modify** data before passing it through.

Used for:

- **Form validation**
- **API request interceptors (Axios)**
- **State control or access restriction**

Example:

```
const user = { name: "swapnil banta", age: 25 };

const userProxy = new Proxy(user, {
 set(obj, prop, value) {
  if (prop === "age" && value < 0) throw new Error("Invalid age");
  console.log(`Setting ${prop} = ${value}`);
  obj[prop] = value;
  return true;
 }
});
```

```
userProxy.age = 30;
```

6. Prototype Pattern

The **Prototype Pattern** allows new objects to be created from an existing "prototype" object, reusing its properties and methods.

Used in class components and object inheritance (React's older OOP model).

Eample:

```
const baseComponent = {
  render() { console.log("Rendering..."); },
};

const Header = Object.create(baseComponent);
Header.render();
```

7. Decorator Pattern

- The **Decorator Pattern** adds extra functionality to existing components **without modifying their code**.

Used as Higher Order Components (HOCs) or wrapper components.

Example:

```
function withLogger(WrappedComponent) {
  return function (props) {
    console.log("Rendered:", WrappedComponent.name);
    return <WrappedComponent {...props} />;
```

```
  };
}

function Hello() { return <h3>Hello World!</h3>; }

export default withLogger(Hello);
```